

Sunwoo Jeong

781-325-5558 | sunwoojeongOfficial@gmail.com
431 Deerfield Ave, Irvine, CA, 92606

<https://www.linkedin.com/in/sunwoo-jeong-4b6621198>
<https://github.com/swjeong0825>

Summary

Software Engineer with 3+ years at Amazon building **backend services** and **data pipelines** on AWS. Skilled in API design and shipping scalable event driven systems using API Gateway, Lambda, DynamoDB, SQL, and infrastructure as code with AWS CDK and Smithy defined contracts. Delivered high throughput production services at scale, including systems handling over 1 million requests per minute with sub 140 millisecond latency. Built ML and LLM driven workflows for catalog quality, including misclassification detection at very large scale and multilingual embedding training using PyTorch and Hugging Face in Docker on EC2.

Skills

- Programming Languages: Python, Java, JavaScript, TypeScript
- Technologies: AWS, Lambda, DynamoDB, FastAPI, React.js, ExpressJS, Prisma ORM, MySQL, PostgreSQL

EXPERIENCES

Amazon.com

Software Developer Engineer

Irvine, CA

July 2022 - Jan 2026

- Built real-time event-driven pipelines with Amazon SQS and Lambda to process live customer events for recommendation systems at scale; also developed backend services with DynamoDB and internal TypeScript frameworks serving 1M+ requests per minute at sub-160ms latency.
- Built a Backend Service on AWS (API Gateway + Lambda + DynamoDB) for product recommendations and metric management. Implemented a contract-first stack with AWS CDK (IaC), Smithy-defined APIs on API Gateway, and Java Lambdas
- Designed a two-stage (ML + LLM) pipeline that pre-detects misclassified item-to-category mappings in a 600M-item/40K-category catalog and publishes a daily stoplist to cut manual remediation
 - Impact: corrected misclassifications for items representing ~2% of total product-page views (US marketplace).
- Expanded an embedding-based item categorization pipeline from US-only to multilingual marketplaces by selecting Hugging Face multilingual text-to-embedding models, containerizing the ML stack (PyTorch) in Docker, publishing to AWS ECR, and running per-marketplace training jobs on EC2
- Redesigned a media categorization data pipeline by migrating from a monolithic system to modular, entity-specific processing flows for non-purchasable content types (e.g., product photos, videos, collections), improving system isolation and reducing cross-entity failures
 - Enabled scalable onboarding of new media types (e.g., live streams, product reviews) with type-specific logic, including computer vision-based categorization for visual content
- Automated S3-to-DynamoDB ingestion with a Lambda listener on S3 events, keeping the metrics dataset current
- Languages/Technologies: Python, Java, AWS SQS, AWS Lambda, DynamoDB, SQL, Docker

Advanced Continuing Education Association (ACEA)

Full Stack Developer Intern

Boston, MA

June 2021 - August 2021

- Backend Data Flow and Query Optimization for Certificate Data Saving Feature with Java Spring
- Languages/Technologies: Java Spring Framework, React.js, MySQL

Projects

Tennis League Management Bot

Live: <https://tlmb.swjapps.com>

- Architected the core REST backend with Domain-Driven Design: modeled League and Match as aggregate roots with value objects, policies, and domain services, layered in clean architecture (domain/application/infrastructure) with a Repository pattern and Unit of Work for transactional consistency
 - https://github.com/swjeong0825/TLMB_backend_main
- Built a chat-to-intent microservice that classifies natural-language chat messages into 10 structured intents using an LLM; implemented a two-stage pipeline — intent classification with per-intent confidence thresholds followed by parameter

extraction — with automatic clarification questions on low-confidence inputs

- https://github.com/swjeong0825/TLMB_chat_to_intent
- Bifurcated READ vs. WRITE intents: READ intents (standings, history, roster) return fetched data directly; WRITE intents (submit score, edit nickname, delete match) return pre-filled form payloads for the frontend to review and confirm — preventing unintended mutations
- Languages/Technologies: Python, FastAPI, PostgreSQL, SQLAlchemy, OpenAI API, Groq API, JavaScript

Auto Bookclub AI

Live: <https://auto-bookclub-ai.vercel.app/>

- Architected a full-stack AI book discussion platform (Next.js App Router) that auto-generates structured debates between two LLM-created contrasting personas; enforced deterministic structured output using OpenAI Responses API with strict JSON Schema validation
 - <https://github.com/swjeong0825/Auto-Bookclub-AI>
- Designed a two-stage LLM pipeline — first generating book-specific personas with opposing reading philosophies, then producing sequential per-turn debate transcripts — with an interactive continuation flow allowing users to inject prompts and extend discussions indefinitely
- Languages/Technologies: TypeScript, Next.js, OpenAI API, Zustand, Open Library API, Google Books API, Vercel, Clerk

EDUCATION

University of Massachusetts, Amherst

Amherst, MA

Graduation: May 2022

- Major: B.S. Computer Science, Minor: Statistics, GPA: 3.97
- Courses For Software Development: Introduction to Software Engineering, Practice and Applications of Data Management, Programming with Data Structures, Computer Systems Principles, Introduction To Algorithms, Leadership Practicum
- Courses For Data Science: Algorithms for Data Science, Principles of Data Science, Artificial Intelligence